notification-service Module 6

Mohamed Taher Elkhouja · Projet MeteoWatch · 4SSA15 – Groupe G3 · AWD 2026-2027 · Validation 1
Dossier individuel : conception (I1), interactions (I2), projet et Git (I3), préparation aux questions (I4)

1. Rôle du service dans MeteoWatch

Quand une alerte de vigilance est déclenchée, **notification-service** prévient automatiquement les utilisateurs abonnés à la région concernée, par email ou push, et garde l'historique de ce qu'ils ont reçu (lue / non lue).

Ce que le service fait (contexte borné : diffusion des messages)	Ce qu'il ne fait pas (autres services)
• Reçoit l'événement `AlertTriggered` • Construit le message à partir d'un modèle • Envoie par email ou push • Historise par utilisateur, lue / non lue • Gère les modèles de messages	• Détecter un dépassement de seuil : **alert-service** • Gérer les abonnements et les profils : **user-service** • Stocker les relevés : **readings-service**

Fiche : port **8086** · Spring Boot · base **MongoDB** (`notification_db`) · enregistré dans Eureka, configuré par le Config Server, accessible uniquement via l'API Gateway (JWT Keycloak).

2. Diagramme de cas d'utilisation

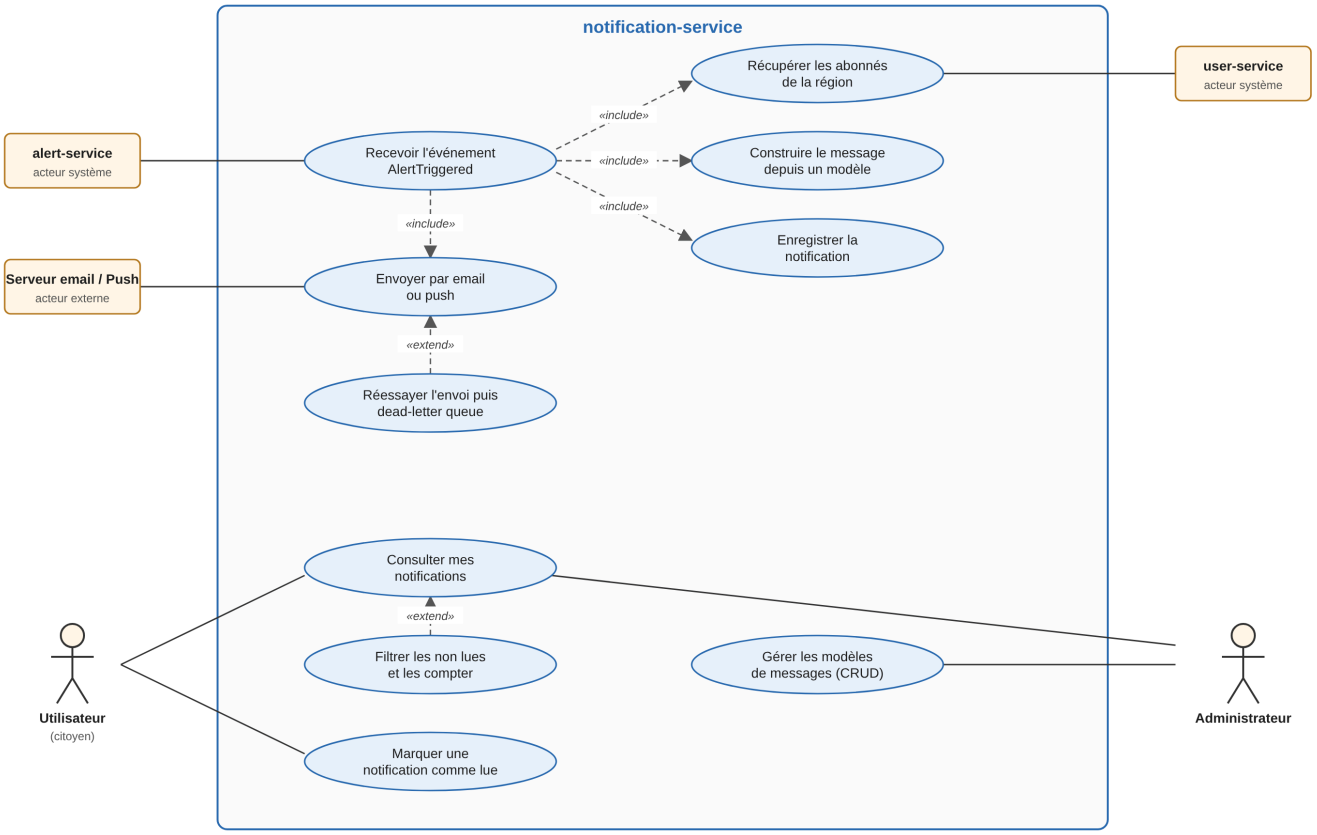


Figure 1 – Cas d'utilisation de notification-service

Acteur	Rôle	Cas d'utilisation
alert-service	Système déclencheur	Publie `AlertTriggered` : recevoir l'événement
user-service	Système appelé	Fournit les abonnés d'une région avec leurs canaux
Serveur email / Push	Système externe	Envoie réellement le message
Utilisateur (citoyen)	Rôle USER	Consulter ses notifications, filtrer les non lues, marquer comme lue
Administrateur	Rôle ADMIN	Gérer les modèles de messages (CRUD), consulter les notifications

3. Conception du microservice (I1)

3.1 Modèle de données et choix de la base

Base **MongoDB** : chaque notification est un document autonome (contenu, canal, statut d'envoi, lu / non lu), sans jointure avec d'autres services. Le volume grandit avec le nombre d'abonnés par alerte, et le contenu est semi-structuré (corps de message variable selon canal et modèle). Index sur `userId` et `createdAt` pour l'historique, et index unique sur (`alertId`, `userId`, `channel`) pour éviter un doublon si RabbitMQ redélivre un message.

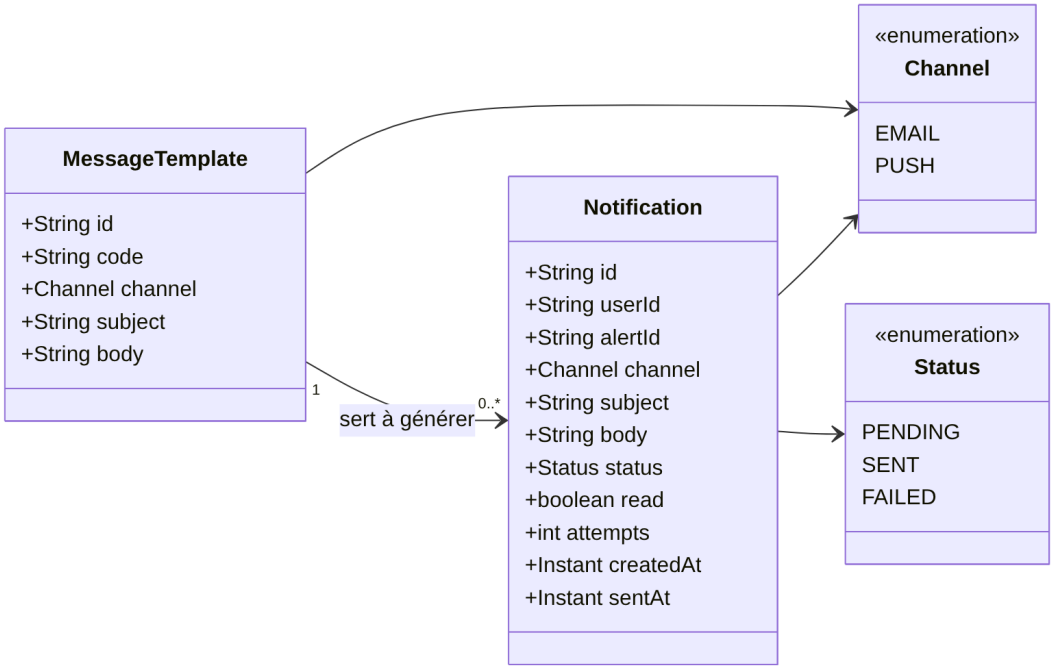


Figure 2 – Diagramme de classes

Entité	Attributs
Notification	id, userId (identifiant Keycloak), alertId, channel (EMAIL, PUSH), subject, body, status (PENDING, SENT, FAILED), read, attempts, createdAt, sentAt
MessageTemplate	id, code, channel, subject, body avec variables `{{region}}`, `{{level}}`, `{{value}}`

```
{ "_id": "66f1...", "userId": "a1b2-keycloak-sub", "alertId": "AL-2026-118",
  "channel": "EMAIL", "subject": "Vigilance ORANGE — Nord",
  "body": "Vent à 95 km/h attendu dans la région Nord.",
  "status": "SENT", "read": false, "attempts": 1, "createdAt": "2026-11-12T08:14:00Z" }
```

3.2 API REST

Méthode	URI	Description	Rôle	Codes
GET	/api/notifications/me	Mes notifications, filtre `?read=false`	USER	200, 401
PATCH	/api/notifications/{id}/read	Marquer comme lue	USER	200, 401, 404
GET	/api/notifications/me/unread-count	Nombre de non lues	USER	200, 401
POST	/api/templates	Créer un modèle	ADMIN	201, 400, 401, 403, 409
GET	/api/templates	Lister les modèles	ADMIN	200, 401, 403
PUT	/api/templates/{id}	Modifier un modèle	ADMIN	200, 400, 403, 404
DELETE	/api/templates/{id}	Supprimer un modèle	ADMIN	204, 403, 404

L'identifiant de l'utilisateur vient toujours du JWT (claim `sub`), jamais de l'URL : un utilisateur ne peut pas lire les notifications d'un autre. Il n'existe pas de `POST /notifications` exposé : les notifications sont créées uniquement par le traitement de l'événement.

4. Interactions du microservice (I2)

Direction	Service	Mode	Données	Si indisponible
Consomme	alert-service	Asynchrone (RabbitMQ)	`AlertTriggered` : alertId, level, region, value	Message conservé dans la file persistante, traité au retour
Appelle	user-service	Synchrone (OpenFeign)	`GET /api/users/subscribers?region=` : abonnés et canaux	Circuit breaker Resilience4j, retry avec délai croissant, puis dead-letter queue
Appelle	Serveur SMTP / push	Synchrone	Message à envoyer	Statut FAILED, nouvelles tentatives, abandon visible dans l'historique
Appelé par	Frontend via Gateway	Synchrone (REST)	Historique, lecture, modèles	Gateway répond 503 si le service est arrêté

> **Pourquoi ces modes ?**
> **Asynchrone depuis alert-service** : alert-service ne doit pas attendre l'envoi des emails, et aucune alerte ne doit être perdue si notification-service est en panne : la file persistante garde le message.
> **Synchrone vers user-service** : j'ai besoin de la liste des abonnés tout de suite pour continuer le traitement.

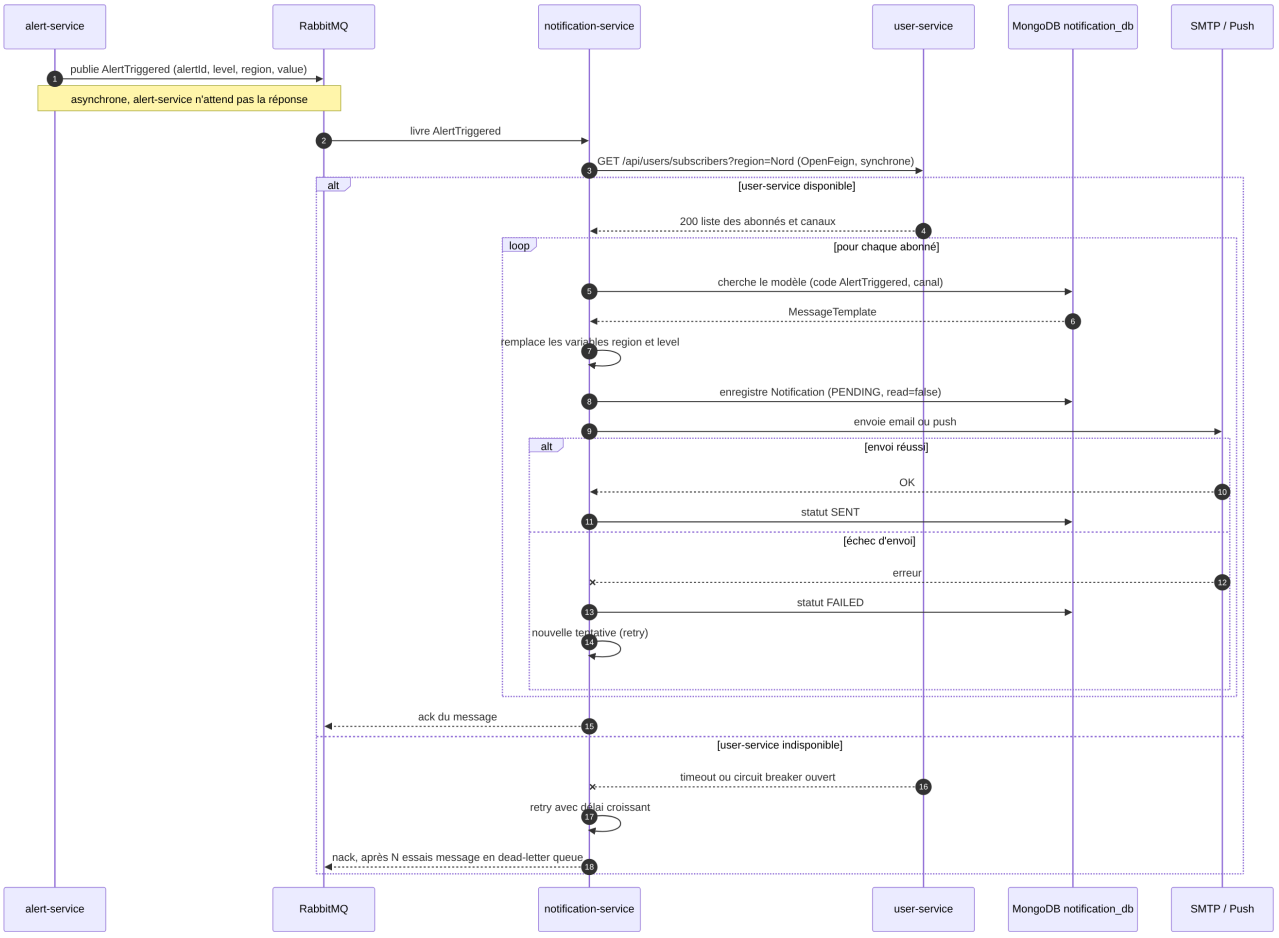


Figure 3 – Séquence : de l'alerte à l'envoi (asynchrone puis synchrone), avec les cas de panne

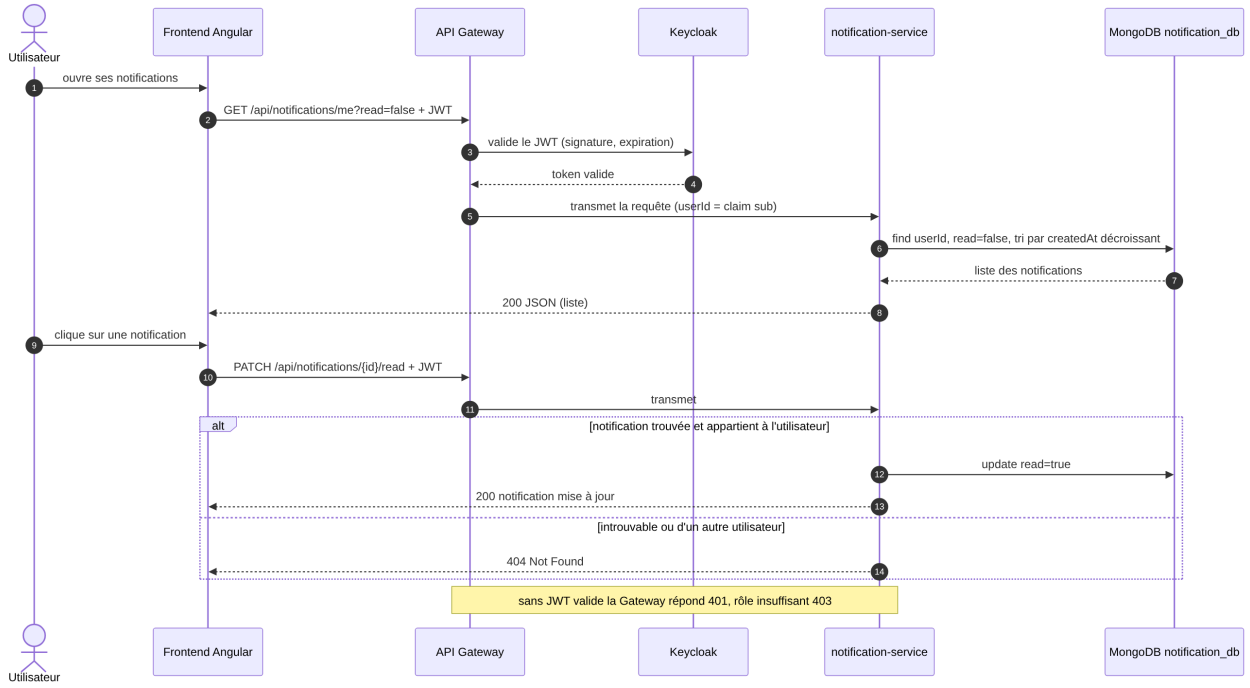


Figure 4 – Séquence : consultation et lecture d'une notification par l'utilisateur

Cas de panne envisagés

Panne	Comportement prévu
user-service arrêté	Retry avec délai croissant, circuit breaker ouvert, après N essais le message va en dead-letter queue pour être rejoué manuellement
Serveur email / push en erreur	Notification marquée FAILED puis nouvelle tentative, le compteur `attempts` limite les essais
notification-service arrêté	Les événements s'accumulent dans RabbitMQ, traités au redémarrage
Message redélivré	L'index unique (alertId, userId, channel) empêche d'envoyer deux fois la même notification

5. Projet créé et contribution Git (I3)

- Dossier : `Backend/notification-service/`, projet Spring Boot (Maven, Java 17+) qui démarre sur le port 8086.
- Dépendances : Spring Web, Spring Data MongoDB, Validation, Eureka Discovery Client, Config Client, OpenFeign, Spring for RabbitMQ, Mail, OAuth2 Resource Server, Resilience4j.
- Pour démarrer sans infrastructure : `eureka.client.enabled=false`, `spring.cloud.config.enabled=false`, MongoDB local (`mongodb://localhost:27017/notification_db`).
- Branches `feature/notification-service-*` vers `develop` par Pull Request. Commits réguliers, par exemple `feat(notification-service): initialisation du projet`.
- Tâche transverse : README principal et document technique.
- Diagrammes versionnés dans `Documentation/Diagrams/notification-service/`.

6. Préparation aux questions (I4)

Pourquoi un microservice séparé pour les notifications ? C'est un contexte borné différent : diffuser des messages. Il a son rythme de charge (un pic d'envois à chaque alerte) et ses dépendances propres (SMTP, push). On peut le faire évoluer ou le mettre à l'échelle sans toucher aux alertes.

Pourquoi MongoDB et pas une base relationnelle ? Documents autonomes, volume croissant, contenu semi-structuré, aucune jointure avec d'autres services. Le choix est équilibré dans l'équipe : 3 services PostgreSQL, 3 MongoDB.

Pourquoi asynchrone avec alert-service ? Pour ne pas bloquer le déclenchement de l'alerte pendant l'envoi des emails et pour ne perdre aucune alerte si mon service est arrêté.

Et si user-service est en panne ? Retry avec délai croissant, circuit breaker Resilience4j, puis dead-letter queue. Le message n'est pas perdu.

Pourquoi ne pas lire les abonnés directement dans la base de user-service ? Chaque service possède sa base : aucune base partagée. On passe par son API (OpenFeign).

Comment éviter les doublons ? Index unique (alertId, userId, channel) en cas de redélivrance RabbitMQ.

Comment sécurisez-vous l'API ? La Gateway valide le JWT Keycloak. L'identifiant utilisateur vient du claim `sub`. Les modèles sont réservés au rôle ADMIN.

À quoi sert Eureka ici ? notification-service s'y enregistre et OpenFeign retrouve user-service par son nom, sans adresse écrite en dur.

À quoi sert l'API Gateway ? Point d'entrée unique : routage vers les services, validation du JWT, aucun service exposé directement.

Quelle différence entre synchrone et asynchrone ? Synchrone : l'appelant attend la réponse (OpenFeign). Asynchrone : l'émetteur publie un événement et continue (RabbitMQ), avec un couplage plus faible.

7. Speech individuel (1 minute)

> « Mon microservice, **notification-service**, prévient les utilisateurs quand une alerte est déclenchée. Il écoute l'événement **AlertTriggered** sur RabbitMQ, demande à **user-service** la liste des abonnés de la région par OpenFeign, construit le message à partir d'un modèle et l'envoie par email ou push. Les données sont dans **MongoDB** : une collection de notifications avec le statut d'envoi et lue / non lue, et une collection de modèles. L'API permet à l'utilisateur de consulter ses notifications et de les marquer comme lues, et à l'administrateur de gérer les modèles. En cas de panne de user-service, j'applique un retry avec circuit breaker puis une dead-letter queue, donc aucune alerte n'est perdue. »